

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Ordinamento e ricerca

Perché ordinare?

Ordinare un array significa disporne gli elementi in ordine crescente (o decrescente).

- **Statistica:** mediana, quantili, valori estremi.
- **Ricerca veloce:** in un elenco ordinato si trova un elemento molto più in fretta (pensate a un dizionario).
- **Presentazione:** tabelle e classifiche leggibili.
- **Confronti:** trovare valori ripetuti è facile, se sono vicini.

Come mettiamo queste particelle in ordine di massa? Ci serve un **algoritmo** di ordinamento.

Particella	Massa [MeV/c^2]
protone	938.27
elettrone	0.511
pione carico	139.57
muone	105.66
kaone carico	493.68
Λ	1115.68
pione neutro	134.98
η	547.86

Scambiare due variabili

L'operazione di base di quasi tutti gli ordinamenti è lo **scambio** (*swap*) di due valori.

```c
#include <stdio.h>

int main() {
    double a = 1.0, b = 2.0;

    a = b;                // SBAGLIATO: il vecchio a è perso
    b = a;
    printf("a = %.1f, b = %.1f\n", a, b);

    a = 1.0; b = 2.0;
    double tmp = a;       // GIUSTO: si salva a
    a = b;
    b = tmp;
    printf("a = %.1f, b = %.1f\n", a, b);
    return 0;
}
```

```
$ ./scambio
a = 2.0, b = 2.0
a = 2.0, b = 1.0
```

- Dopo `a = b;` il vecchio valore di `a` è **sovrascritto**: non si recupera più.
- Serve una variabile **temporanea**, come per scambiare il contenuto di due bicchieri serve un terzo bicchiere.
- Con gli elementi di un array è identico:

```c
double tmp = v[j];
v[j] = v[j + 1];
v[j + 1] = tmp;
```

Bubble sort: l'idea

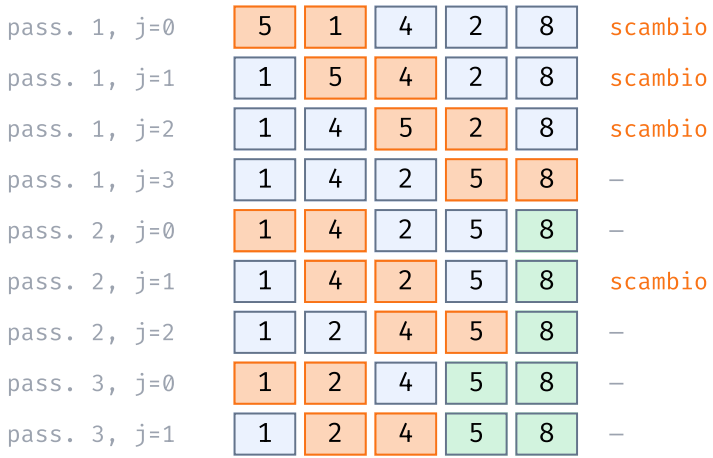
- Si scorre l'array confrontando gli elementi **adiacenti** `v[j]` e `v[j + 1]`.
- Se sono nell'ordine sbagliato (`v[j] > v[j + 1]`), si **scambiano**.
- Alla fine della prima **passata**, il massimo è "salito" fino all'ultima posizione, come una **bolla** in un bicchiere.
- Alla seconda passata sale il secondo più grande, e così via.
- Dopo $N - 1$ passate l'array è **ordinato**.

In pseudocodice

```
per i da 0 a N-2:                  (passate)
    per j da 0 a N-2-i:            (confronti)
        se v[j] > v[j+1]:
            scambia v[j] e v[j+1]
```

- A ogni passata un elemento in più è già al suo posto in fondo: il ciclo interno si **accorcia** (`- i`).
- Algoritmo semplice, ma **lento** per array grandi (vedremo quanto).

Bubble sort passo per passo



- In arancione la coppia confrontata, in verde gli elementi già al loro posto.
- **Passata 1:** l' `8` era già in fondo, ma il `5` "sale" fino alla penultima posizione.
- **Passata 2:** un solo scambio, `4` ↔ `2`; ora l'array è ordinato.
- **Passata 3:** nessuno scambio. Senza ulteriori accorgimenti l'algoritmo farebbe comunque anche la passata 4.

Bubble sort in C

```c
#include <stdio.h>

#define N 8

int main() {
    double m[N] = {938.27, 0.511, 139.57, 105.66,
                   493.68, 1115.68, 134.98, 547.86};   // [MeV/c^2]
    int i, j;

    for (i = 0; i < N - 1; i++) {             // passate
        for (j = 0; j < N - 1 - i; j++) {     // confronti
            if (m[j] > m[j + 1]) {
                double tmp = m[j];
                m[j] = m[j + 1];
                m[j + 1] = tmp;
            }
        }
    }
    for (i = 0; i < N; i++) {
        printf("%8.3f\n", m[i]);
    }
    return 0;
}
```

```
$ ./bubble
   0.511
 105.660
 134.980
 139.570
 493.680
 547.860
 938.270
1115.680
```

- `j < N - 1 - i`: così `m[j + 1]` non esce **mai** dall'array.
- Per l'ordine **decrescente** basta cambiare `>` in `<`.
- **Test:** un array già ordinato deve restare uguale; uno al contrario deve risultare capovolto.

Quanti confronti?

- Passata 1: $N - 1$ confronti; passata 2: $N - 2$; …; ultima: 1.
- Totale:

$$(N-1) + (N-2) + \cdots + 1 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$$

- Si dice che il bubble sort ha complessità $O(N^2)$: raddoppiando N il tempo **quadruplica**.

Esistono algoritmi molto più veloci (*quicksort*, *mergesort*), che richiedono circa $N \log_2 N$ confronti: per $N = 10^6$, 2×10^7 invece di 5×10^{11}!

N	confronti contati	tempo (portatile)
10	45	
100	4 950	
1 000	499 500	
10 000	49 995 000	0.4 s
100 000	4 999 950 000	26 s
1 000 000	$\approx 5 \times 10^{11}$	≈ 45 min (stima)

Confronti contati con un contatore `confronti++` nel ciclo interno, su numeri casuali (lezione 19). I tempi dipendono dalla macchina.

Ottimizzazione: fermarsi quando è ordinato

```
1   for (i = 0; i < N - 1; i++) {
2       scambiato = 0;
3       for (j = 0; j < N - 1 - i; j++) {
4           confronti++;
5           if (v[j] > v[j + 1]) {
6               double tmp = v[j];
7               v[j] = v[j + 1];
8               v[j + 1] = tmp;
9               scambiato = 1;
10          }
11      }
12      if (!scambiato) {
13          break;          // nessuno scambio: già ordinato
14      }
15  }
```

- `scambiato` è una **bandierina** (*flag*): vale `1` se nella passata c'è stato almeno uno scambio.
- Se una passata intera non scambia nulla, l'array è **ordinato**: `break` (lezione 15).

Array iniziale ($N = 1000$)	confronti
casuale	498 225
già ordinato	999
ordinato al contrario	499 500

- Il caso tipico **non** migliora molto; quello quasi ordinato moltissimo.

Applicazione: la mediana

La **mediana** è il valore centrale dei dati **ordinati**: metà dei dati è sotto, metà sopra.

```c
double t[N] = {0.452, 0.448, 0.455, 0.451, 0.449,
               0.639, 0.453, 0.450, 0.454};   // tempi di caduta [s]
double somma = 0.0, mediana;

for (i = 0; i < N; i++) {
    somma += t[i];
}
// ... qui il bubble sort di t[] (visto prima) ...

if (N % 2 == 1) {
    mediana = t[N / 2];
} else {
    mediana = (t[N / 2 - 1] + t[N / 2]) / 2.0;
}
printf("media   = %.4f s\n", somma / N);
printf("mediana = %.4f s\n", mediana);
```

```
$ ./mediana
media   = 0.4723 s
mediana = 0.4520 s
```

- Caduta da $h = 1.00$ m: atteso $\sqrt{2h/g} = 0.4515$ s.
- `0.639` è **anomala** (cronometro fermato in ritardo): sposta la media di 20 ms.
- La mediana **non** se ne accorge: è una stima **robusta**.
- L'ordinamento **modifica** l'array originale!

Ricerca lineare

Trovare il **primo** elemento che soddisfa una condizione: si guardano gli elementi **uno alla volta**.

```c
#include <stdio.h>

#define N 12

int main() {
    double s[N] = {0.02, 0.05, 0.01, 0.04, 0.03, 0.08,
                   0.61, 0.95, 0.72, 0.40, 0.18, 0.07};   // [V]
    const double soglia = 0.50;                           // [V]
    int i, pos = -1;

    for (i = 0; i < N; i++) {
        if (s[i] > soglia) {
            pos = i;
            break;      // il primo basta: si esce dal ciclo
        }
    }
    if (pos >= 0) {
        printf("soglia superata al campione %d (%.2f V)\n",
               pos, s[pos]);
    } else {
        printf("soglia mai superata\n");
    }
```

```
$ ./soglia
soglia superata al campione 6 (0.61 V)
```

- Segnale campionato di un rivelatore: un **trigger** scatta al superamento della soglia.
- `pos = -1` vuol dire "non trovato": nessun indice valido è negativo.
- Caso peggiore: N confronti.
- Funziona **sempre**, anche su dati non ordinati.

Ricerca binaria: l'idea

Su un array **ordinato**: si guarda l'elemento **centrale** e si scarta metà dell'array. Cerchiamo `61` tra i primi:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
passo 1	2	3	5	7	11	13	17	19	23	29	31	37	**41**	43	47	53	59	61	67	71	73	79	83	89	97
passo 2	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	61	**67**	71	73	79	83	89	97
passo 3	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	**53**	59	61	67	71	73	79	83	89	97
passo 4	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	**59**	61	67	71	73	79	83	89	97
passo 5	2	3	5	7	11	13	17	19	23	29	31	37	41	43	47	53	59	**61**	67	71	73	79	83	89	97

- L'elemento può stare solo in `[lo, hi]`; si guarda `mid = (lo + hi) / 2`.
- `p[mid] < x`: è a **destra**, `lo = mid + 1`.
- `p[mid] > x`: è a **sinistra**, `hi = mid - 1`.

- Trovato in **5** passi invece di 18.
- Se `lo > hi` l'elemento **non c'è**.
- Come la **bisezione** per gli zeri (lezione 16).

Ricerca binaria in C

```c
int p[N] = { 2,  3,  5,  7, 11, 13, 17, 19, 23, 29, 31, 37, 41,
             43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97};
int x, lo = 0, hi = N - 1, pos = -1;
...
while (lo <= hi) {
    int mid = (lo + hi) / 2;
    printf("lo = %2d  hi = %2d  mid = %2d  p[mid] = %d\n",
           lo, hi, mid, p[mid]);
    if (p[mid] == x) {
        pos = mid;
        break;
    } else if (p[mid] < x) {
        lo = mid + 1;       // x è nella metà destra
    } else {
        hi = mid - 1;       // x è nella metà sinistra
    }
}
```

Prima del ciclo si legge `x` con `scanf`; dopo, si stampa il risultato in base a `pos`.

```
$ ./binaria
Numero da cercare: 61
lo =  0  hi = 24  mid = 12  p[mid] = 41
lo = 13  hi = 24  mid = 18  p[mid] = 67
lo = 13  hi = 17  mid = 15  p[mid] = 53
lo = 16  hi = 17  mid = 16  p[mid] = 59
lo = 17  hi = 17  mid = 17  p[mid] = 61
61 è primo: p[17]
```

```
$ ./binaria
Numero da cercare: 60
...                 (stessi 5 passi)
60 non è primo
```

- La condizione è `lo <= hi`: con `<` si perderebbe il caso `lo == hi` (l'ultimo passo qui sopra!).
- `mid` è intero: la divisione **intera** arrotonda per difetto.

Quanti passi?

- A ogni passo l'intervallo si **dimezza**: dopo k passi restano circa $N/2^k$ elementi.
- Ci si ferma quando ne resta uno: $N/2^k \approx 1$, cioè

$$k \approx \log_2 N$$

- La ricerca lineare richiede fino a N passi.

Ordinare costa più di una ricerca lineare: conviene se poi si fanno **molte** ricerche sugli stessi dati.

N	lineare	binaria	$\log_2 N$
1 000	1 000	10	9.97
1 000 000	1 000 000	20	19.93
10^9	10^9	30	29.9

Passi contati da un programma, nel caso peggiore (elemento assente), per $N = 10^3$ e 10^6; la riga 10^9 è una stima.

- Passando da mille a un milione di elementi, la ricerca binaria fa solo **10 passi in più**!

Errori tipici

Errore	Cosa succede
Scambio senza variabile temporanea	Si perde uno dei due valori: due copie dello stesso numero
Ciclo interno `j < N - i` (o `j < N`)	`v[j + 1]` esce dall'array all'ultimo confronto
`>` al posto di `<` (o viceversa)	Ordine crescente e decrescente invertiti: fate un test!
Ricerca binaria su un array **non** ordinato	Risultati sbagliati, senza alcun avviso
`while (lo < hi)` nella ricerca binaria	Non si controlla l'ultimo elemento rimasto